

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэкенд-разработка

Отчёт

Лабораторная работа №3
«Контейнеризация написанного приложения
средствами docker»

Выполнил:
Клименков Владислав

Группа:
К3341

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задачи

Реализовать Dockerfile для каждого микросервиса, написать общий docker-compose.yml, настроить сетевое взаимодействие между сервисами.

Ход работы

1. Dockerfile для каждого сервиса

1.1 API Gateway (`services/api-gateway/Dockerfile`)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

API Gateway — stateless сервис, ему не требуется Prisma generate (нет своей БД).

1.2 User Service (`services/user/Dockerfile`)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY prisma ./prisma
RUN npx prisma generate
COPY . .
EXPOSE 3001
CMD ["npm", "start"]
```

User Service требует генерации Prisma Client перед копированием исходного кода, так как Prisma Client генерируется на основе схемы.

1.3 Recipe Service (`services/recipe/Dockerfile`)

```
FROM node:20-alpine
WORKDIR /app
```

```
COPY package*.json ./
RUN npm install
COPY prisma ./prisma
RUN npx prisma generate
COPY . .
EXPOSE 3002
CMD ["npm", "start"]
```

Аналогично User Service, требуется генерация Prisma Client.

2. Docker Compose (docker-compose.yml)

```
services:
  rabbitmq:
    image: rabbitmq:4.1-management-alpine
    container_name: recipe-hub-rabbitmq
    ports:
      - "5672:5672"
      - "15672:15672"
    environment:
      RABBITMQ_DEFAULT_USER: guest
      RABBITMQ_DEFAULT_PASS: guest
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics",
"check_port_connectivity"]
      interval: 10s
      timeout: 5s
      retries: 5

  user-db:
    image: postgres:18.3-alpine
    container_name: user-db
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: user_db
      PGDATA: /var/lib/postgresql/data
    ports:
      - "5433:5432"
    volumes:
      - user_db_data:/var/lib/postgresql/data
    healthcheck:
```

```

    test: ["CMD-SHELL", "pg_isready -U user -d user_db"]
    interval: 10s
    timeout: 5s
    retries: 5

recipe-db:
  image: postgres:18.3-alpine
  container_name: recipe-db
  environment:
    POSTGRES_USER: user
    POSTGRES_PASSWORD: password
    POSTGRES_DB: recipe_db
    PGDATA: /var/lib/postgresql/data
  ports:
    - "5434:5432"
  volumes:
    - recipe_db_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U user -d recipe_db"]
    interval: 10s
    timeout: 5s
    retries: 5

recipe-service:
  build: ./services/recipe
  env_file:
    - ./services/recipe/.env
  depends_on:
    recipe-db:
      condition: service_healthy
    rabbitmq:
      condition: service_healthy
  restart: on-failure

user-service:
  build: ./services/user
  env_file:
    - ./services/user/.env
  depends_on:
    user-db:
      condition: service_healthy
    rabbitmq:

```

```

        condition: service_healthy
    restart: on-failure

api-gateway:
    build: ./services/api-gateway
    ports:
        - "3000:3000"
    env_file:
        - ./services/api-gateway/.env
    depends_on:
        user-service:
            condition: service_started
        recipe-service:
            condition: service_started
    restart: on-failure

prisma-studio-user:
    build: ./services/user
    container_name: prisma-studio-user
    command: ["npx", "prisma", "studio", "--port", "5555"]
    ports:
        - "5555:5555"
    environment:
        - BROWSER=none
    env_file:
        - ./services/user/.env
    depends_on:
        user-db:
            condition: service_healthy
    profiles:
        - prisma-studio

prisma-studio-recipe:
    build: ./services/recipe
    container_name: prisma-studio-recipe
    command: ["npx", "prisma", "studio", "--port", "5556"]
    ports:
        - "5556:5556"
    environment:
        - BROWSER=none
    env_file:
        - ./services/recipe/.env

```

```

depends_on:
  recipe-db:
    condition: service_healthy
profiles:
  - prisma-studio

volumes:
  user_db_data:
  recipe_db_data:

```

3. Сетевое взаимодействие

Docker Compose автоматически создаёт сеть `recipe-hub\_default`, в которой все сервисы доступны по именам контейнеров:

Имя в Docker	Доступен по адресу
rabbitmq	`amqp://guest:guest@rabbitmq:5672`
user-db	`postgresql://user:password@user-db:5432/user_db`
recipe-db	`postgresql://user:password@recipe-db:5432/recipe_db`
user-service	`http://user-service:3001`
recipe-service	`http://recipe-service:3002`
api-gateway	`http://localhost:3000` (с хоста)

Вывод

Все три микросервиса (API Gateway, User Service, Recipe Service) успешно контейнеризированы с помощью Docker. Каждый сервис имеет собственный Dockerfile с оптимизированной многослойной сборкой. Общий docker-compose.yml описывает инфраструктуру из 6 контейнеров: 2 базы данных PostgreSQL, RabbitMQ, 3 микросервиса. Настроено сетевое взаимодействие через внутреннюю Docker-сеть, healthcheck для зависимостей, тома для персистентности данных. Приложение запускается одной командой `docker compose up --build -d`.